

10 учебных проектов с ИИ через API

Подборка заданий на разработку небольших проектов, использующих LLM-API. Каждое задание можно выполнить за 1–3 дня и собрать в портфолио. Список бесплатных API-сервисов — в конце документа.

Задания

1. Telegram-бот для саммаризации статей

Цель. Научиться объединять внешний API (LLM) с входным веб-контентом и мессенджером.

Что делает проект. Пользователь отправляет в бот ссылку на статью. Бот скачивает страницу, извлекает основной текст (например, через `readability` / `trafilatura`) и возвращает краткое содержание в 5 пунктах.

Технические требования:

- Python (`python-telegram-bot` / `aiogram`) или Node.js (`grammy`).
- Извлечение текста из HTML.
- Обработка ошибок (битые ссылки, пэйволы, слишком длинный текст).

Что осваивает студент: работа с REST API, парсинг HTML, ограничения контекста LLM, чанкинг длинных текстов.

2. CLI-инструмент для генерации commit-сообщений

Цель. Понять, как ИИ интегрируется в повседневный инструментарий разработчика.

Что делает проект. Утилита запускается в git-репозитории, берёт `git diff --staged`, отправляет его в LLM и возвращает осмысленный commit message в формате Conventional Commits (`feat:`, `fix:`, `refactor:` и т.д.).

Технические требования:

- Любой язык (Python, Go, Rust, Node.js).

- Чтение вывода `git diff` через `subprocess`.
- Флаги: `--type`, `--scope`, `--dry-run`.

Что осваивает студент: проектирование промптов под структурированный вывод, работа с CLI-аргументами, упаковка приложения.

3. Ассистент для заметок в Obsidian / markdown-папке

Цель. Построить простую систему знаний с помощью LLM.

Что делает проект. Скрипт сканирует папку с `.md`-файлами, находит смысловые связи между заметками и предлагает добавить ссылки между ними. Опционально — генерирует краткое summary каждой заметки во фронтматтер.

Технические требования:

- Чтение файлов рекурсивно.
- Использование `embeddings API` для поиска похожих заметок.
- Не модифицировать файлы без подтверждения пользователя.

Что осваивает студент: векторные представления текста (`embeddings`), косинусная близость, безопасная работа с пользовательскими файлами.

4. Генератор unit-тестов по коду

Цель. Понять, как LLM применяются для автоматизации тестирования.

Что делает проект. На вход подаётся файл с функцией (Python/JS), на выходе — набор unit-тестов в `pytest` / `jest`. Тесты должны покрывать `happy path`, `edge cases` и обработку ошибок.

Технические требования:

- AST-парсинг для извлечения сигнатур функций.
- Сохранение тестов в отдельный файл.
- Проверка, что сгенерированные тесты вообще запускаются.

Что осваивает студент: структурированные промпты, AST, валидация вывода LLM.

5. RAG-поиск по документации

Цель. Реализовать классический паттерн Retrieval-Augmented Generation.

Что делает проект. Пользователь указывает папку с `.md` или `.pdf` файлами. Скрипт разбивает документы на чанки, считает embeddings, сохраняет в локальную векторную БД (Chroma, FAISS, SQLite-VSS). При запросе — ищет релевантные чанки и передаёт их в LLM как контекст.

Технические требования:

- Чанкинг с overlap.
- Хранение векторов локально.
- Указание источников в ответе.

Что осваивает студент: ключевой паттерн современной разработки с ИИ — RAG; работа с векторными БД.

6. Переводчик markdown-файлов с сохранением форматирования

Цель. Решить нетривиальную задачу — перевод текста, не ломая структуру документа.

Что делает проект. Берёт `.md` или `.mdx` файл, переводит человекочитаемый текст и заголовки, но **не трогает** код в блоках, фронтматтер, ссылки и HTML-теги.

Технические требования:

- Парсинг markdown (например, `markdown-it`, `remark`).
- Раздельная обработка узлов AST.
- Поддержка минимум 3 целевых языков.

Что осваивает студент: работа с AST, аккуратное промптирование при необходимости сохранить структуру.

7. Анализатор скриншотов багов (vision-модель)

Цель. Поработать с мультимодальными моделями.

Что делает проект. Web-приложение или Telegram-бот, куда пользователь загружает скриншот ошибки (терминал, IDE, браузер). Vision-модель описывает проблему,

классифицирует её и предлагает шаги решения.

Технические требования:

- Загрузка изображения, кодирование в base64.
- API с поддержкой vision (Gemini, GPT-4o, Claude).
- Простой UI (Streamlit / минимальный фронт).

Что осваивает студент: мультимодальные API, обработка изображений.

8. Voice-to-task: голосовая заметка → структурированная задача

Цель. Объединить speech-to-text и LLM в одном пайплайне.

Что делает проект. Пользователь записывает голосовую заметку. Whisper (или аналог) транскрибирует. LLM превращает текст в структурированную задачу: заголовок, описание, теги, приоритет, дедлайн. Результат сохраняется в JSON / Notion / Todoist.

Технические требования:

- Запись или загрузка аудио.
- API для транскрипции (Whisper через OpenAI/Groq, либо локальный whisper.cpp).
- LLM с режимом structured output / JSON mode.

Что осваивает студент: связка нескольких моделей в один пайплайн, structured output.

9. AI-модерация комментариев

Цель. Решить production-подобную задачу классификации.

Что делает проект. Сервис (Express / FastAPI / Cloudflare Worker), который принимает текст комментария и возвращает метку: `ok` / `spam` / `toxic` / `needs_review`. Можно добавить веб-интерфейс с историей решений.

Технические требования:

- POST-эндпоинт.
- Логирование решений в БД (SQLite / D1).

- Тесты на пограничные случаи.

Что осваивает студент: API-сервисы, классификация через LLM, оценка качества модели.

10. Подбор GitHub-issues под стек разработчика

Цель. Связать несколько внешних API в одно полезное приложение.

Что делает проект. Пользователь указывает свой GitHub-username. Скрипт анализирует его репозитории через GitHub API, определяет используемые языки и фреймворки, ищет открытые issues с лейблом `good-first-issue` в проектах с похожим стеком и ранжирует их по релевантности с помощью LLM.

Технические требования:

- GitHub REST API или GraphQL.
- Кэширование (issues — большой объём данных).
- Финальный список — топ-10 с обоснованием релевантности.

Что осваивает студент: интеграция нескольких API, ранжирование, управление токенным бюджетом.

Бесплатные API-сервисы для проектов

Все провайдеры ниже не требуют кредитную карту для регистрации (на момент апреля 2026). Лимиты могут меняться — всегда проверяйте актуальную документацию.

Google AI Studio (Gemini) — рекомендуется по умолчанию

- **Модель:** Gemini 2.5 Flash и другие.
- **Лимиты:** ~1500 запросов/день, контекст 1M токенов, поддержка vision.
- **Подходит для:** RAG, переводчик, анализ скриншотов, любые задачи с длинным контекстом.
- **Сайт:** aistudio.google.com
- **Важно:** в EU/UK/Switzerland бесплатный тариф ограничен или недоступен.

Groq — самый быстрый инференс

- **Модели:** Llama 3.3 70B, Mixtral, Gemma и другие open-source.
- **Лимиты:** ~14 400 запросов/день для большинства моделей; жёсткий лимит токенов в минуту.
- **Подходит для:** интерактивных ботов, voice-to-task, любых задач с требованием низкой задержки.
- **Сайт:** console.groq.com
- **API:** OpenAI-совместимый.

OpenRouter — единый шлюз ко множеству моделей

- **Модели:** десятки бесплатных моделей (Llama, Qwen, DeepSeek, Gemini Flash и др.).
- **Лимиты:** 200 запросов/день у бесплатных моделей; после одноразового пополнения на \$10 — 1000 запросов/день.
- **Подходит для:** прототипирования, сравнения моделей, проектов, где нужна гибкость.
- **Сайт:** openrouter.ai

Cloudflare Workers AI

- **Модели:** Llama, Mistral, embeddings-модели.
- **Лимиты:** ежедневная бесплатная квота “neurons”, встроенная интеграция с Workers, D1, Vectorize.
- **Подходит для:** проектов на Cloudflare-инфраструктуре (модерация, RAG через Vectorize).
- **Сайт:** developers.cloudflare.com/workers-ai

GitHub Models

- **Модели:** 45+ моделей (GPT, Llama, Phi, Mistral и др.) для прототипирования.
- **Лимиты:** на запрос — 8K вход / 4K выход; общий лимит — несколько тысяч запросов в день.
- **Подходит для:** студентов, у которых уже есть GitHub-аккаунт. Самый низкий порог входа.
- **Сайт:** github.com/marketplace/models

Hugging Face Inference API

- **Модели:** тысячи open-source моделей.
- **Лимиты:** ~2000 запросов/день, ограничения по конкурентности.
- **Подходит для:** экспериментов с нишевыми моделями, embeddings.
- **Сайт:** huggingface.co/inference-api

Cerebras — быстрый batch-инференс

- **Модели:** Llama, Qwen.
- **Лимиты:** ~1M токенов/день.
- **Подходит для:** массовой обработки документов.
- **Сайт:** cloud.cerebras.ai

Mistral La Plateforme

- **Модели:** Mistral Small, Codestral и другие.
- **Лимиты:** есть бесплатный экспериментальный тариф.
- **Сайт:** console.mistral.ai

Рекомендации для студентов

1. Начните с **Google AI Studio** или **GitHub Models** — самый низкий порог входа.
2. **Используйте OpenAI-совместимый SDK** (`openai Python` / `openai Node.js`) — большинство бесплатных провайдеров его поддерживают, и в случае исчерпания лимитов можно сменить провайдера в одну строчку.
3. **Кэшируйте ответы** — особенно при разработке. Это экономит лимиты и ускоряет итерацию.
4. **Никогда не коммитьте API-ключи** в git. Используйте `.env` и `.gitignore`.
5. **Стэкайте провайдеров.** Когда упрётесь в лимиты одного — добавьте второй как fallback. Это типичный приём в production.

Полезные ссылки

- Список всех бесплатных LLM API: github.com/cheahjs/free-llm-api-resources
- Список постоянно бесплатных API: github.com/mnfst/awesome-free-llm-apis